

Assignment 1 : ANN Fundamentals – Building from Scratch

Implementing Artificial Neural Networks from Scratch using NumPy for Iris Flower Classification

Submitted By:

Rudra Chauhan

Course:

Deep Learning

Date: 07-06-2026

1. Introduction

Artificial Neural Networks (ANNs) are computational models inspired by the structure and functioning of the human brain. They consist of interconnected neurons organized into layers, where each neuron processes input data and passes the result to the next layer. ANNs are widely used in machine learning and deep learning applications such as image classification, speech recognition, natural language processing, and predictive analytics.

The objective of this assignment is to understand the fundamental concepts of neural networks by implementing a Multi-Layer Perceptron (MLP) from scratch using NumPy without relying on deep learning frameworks such as TensorFlow or PyTorch. Building the network from scratch provides a deeper understanding of how neural networks learn through forward propagation, backpropagation, activation functions, loss computation, and gradient descent optimization.

In this project, the Iris Flower Dataset was used to perform a multiclass classification task. The dataset contains measurements of iris flowers belonging to three different species: Setosa, Versicolor, and Virginica. A neural network architecture consisting of an input layer, two hidden layers, and an output layer was designed and trained to classify flower species based on their features. The performance of the model was evaluated using accuracy, confusion matrix, classification report, and various visualizations.

2. Dataset Description

The Iris Dataset is one of the most widely used datasets in machine learning and pattern recognition. It was introduced by Ronald A. Fisher in 1936 and is commonly used for classification tasks and algorithm evaluation.

The dataset contains a total of 150 samples of iris flowers belonging to three different species:

- Iris Setosa
- Iris Versicolor
- Iris Virginica

Each sample is described using four numerical features:

1. Sepal Length (cm)
2. Sepal Width (cm)
3. Petal Length (cm)
4. Petal Width (cm)

The target variable represents the flower species, resulting in a multiclass classification problem with three classes. Each class contains 50 samples, making the dataset balanced and suitable for classification experiments.

In this project, the dataset was loaded using the `load_iris()` function available in the Scikit-learn library. The dataset consists of 150 observations and 4 input features, while the output consists of 3 distinct classes. The goal of the neural network is to learn the relationship between the flower measurements and their corresponding species and accurately classify unseen samples.

Dataset Summary:

Attribute	Value
Dataset Name	Iris Dataset
Total Samples	150
Number of Features	4
Number of Classes	3
Class Labels	Setosa, Versicolor, Virginica

Attribute	Value
Task Type	Multiclass Classification
Source	Scikit-learn / UCI Machine Learning Repository

3. Data Preprocessing

Data preprocessing is an essential step in machine learning and deep learning as it prepares the dataset for efficient training and improves model performance. Several preprocessing techniques were applied to the Iris dataset before training the neural network.

3.1 Dataset Splitting

The dataset was divided into three subsets:

- Training Set: 70%
- Validation Set: 15%
- Test Set: 15%

The training set was used to train the neural network, the validation set was used to monitor model performance during training, and the test set was used to evaluate the final model.

3.2 Feature Standardization

The input features have different ranges and units. To ensure that all features contribute equally during training, Z-score standardization was applied using the StandardScaler technique.

Standardization transforms each feature to have:

- Mean = 0
- Standard Deviation = 1

This improves training stability and helps the neural network converge more efficiently.

3.3 One-Hot Encoding

The target labels were converted into one-hot encoded vectors because the output layer uses the Softmax activation function for multiclass classification.

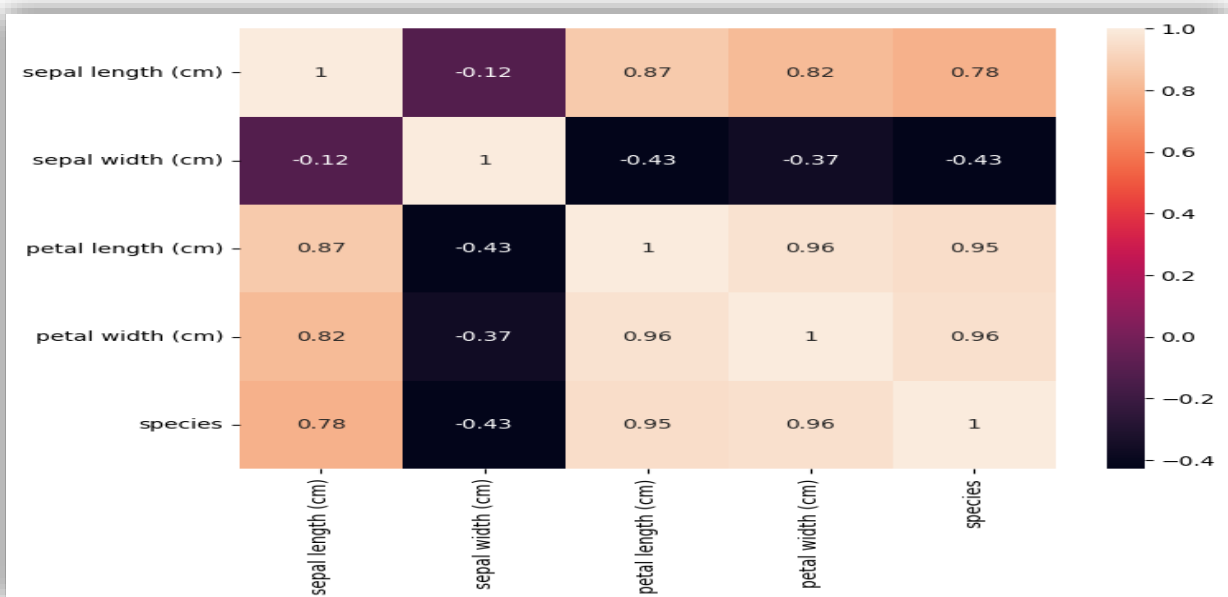
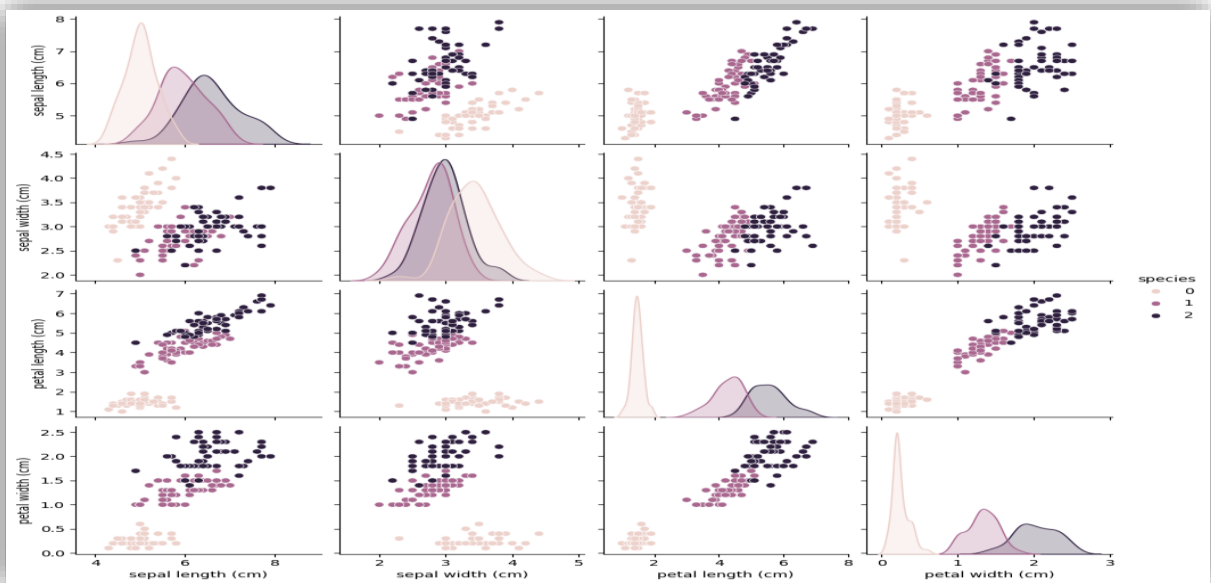
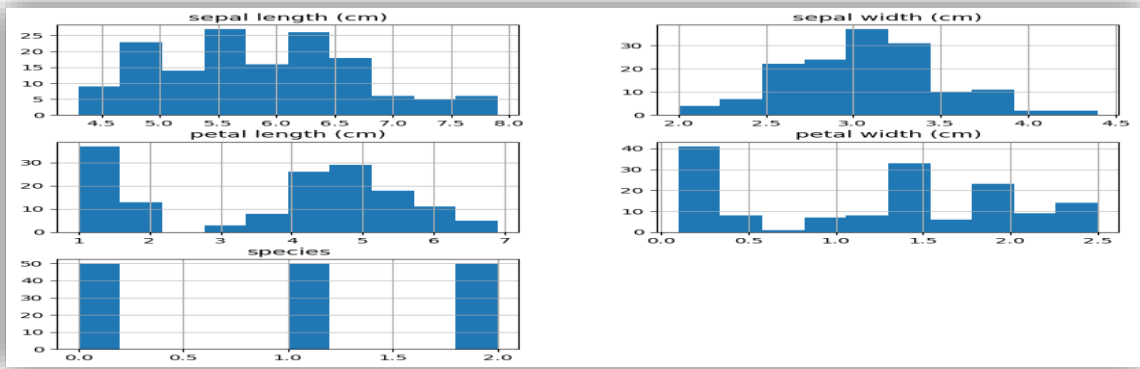
Examples:

- Class 0 → [1, 0, 0]
- Class 1 → [0, 1, 0]
- Class 2 → [0, 0, 1]

One-hot encoding allows the neural network to represent class labels in a format suitable for computing multiclass cross-entropy loss.

4.) Exploratory Data Analysis (EDA)

- Petal length and petal width show strong correlation.
- Setosa species is clearly separable from the other two classes.
- No missing values were present in the dataset.
- The dataset is balanced with 50 samples per class.



5. Neural Network Architecture

A Multi-Layer Perceptron (MLP) neural network was implemented from scratch using NumPy for the classification of Iris flower species. The network consists of an input layer, two hidden layers, and an output layer.

Network Architecture

- Input Layer: 4 neurons
- Hidden Layer 1: 8 neurons with Sigmoid activation
- Hidden Layer 2: 6 neurons with Sigmoid activation
- Output Layer: 3 neurons with Softmax activation

The four input neurons correspond to the four features present in the Iris dataset:

1. Sepal Length
2. Sepal Width
3. Petal Length
4. Petal Width

The hidden layers are responsible for learning complex patterns and relationships within the data. Sigmoid activation functions were used in the hidden layers to introduce non-linearity into the network.

The output layer contains three neurons, representing the three Iris flower species. A Softmax activation function was used in the output layer to convert the network outputs into probability distributions across the three classes.

Network Flow

Input Layer (4 Neurons)

↓

Hidden Layer 1 (8 Neurons, Sigmoid)

↓

Hidden Layer 2 (6 Neurons, Sigmoid)

↓

Output Layer (3 Neurons, Softmax)

Weight and Bias Initialization

The weights were initialized randomly using NumPy, while biases were initialized to zero. Proper initialization helps the neural network learn meaningful patterns and prevents neurons from producing identical outputs.

Activation Functions

The Sigmoid activation function was used in the hidden layers because it maps values between 0 and 1 and allows the network to learn non-linear relationships.

The Softmax activation function was used in the output layer because it is suitable for multiclass classification problems and produces probability values whose sum equals 1.

6. Mathematical Implementation

The neural network was implemented from scratch using NumPy. The model performs forward propagation to generate predictions, computes loss using Cross-Entropy Loss, and updates parameters through backpropagation and gradient descent.

6.1 Forward Propagation

During forward propagation, the input data passes through two hidden layers and one output layer. Each layer computes a weighted sum of inputs followed by an activation function.

- Hidden Layers: Sigmoid Activation
- Output Layer: Softmax Activation

6.2 Sigmoid Activation Function

The Sigmoid activation function was used in the hidden layers to introduce non-linearity and map values between 0 and 1.

Formula:

$$\sigma(x) = 1 / (1 + e^{-x})$$

6.3 Softmax Activation Function

The Softmax activation function was used in the output layer to convert outputs into class probabilities for multiclass classification.

Formula:

$$\text{Softmax}(z_i) = e^{z_i} / \sum e^{z_j}$$

6.4 Cross-Entropy Loss

Cross-Entropy Loss was used to measure the difference between predicted probabilities and actual class labels. Lower loss values indicate better model performance.

Formula:

$$L = - \sum y \log(\hat{y})$$

6.5 Backpropagation and Gradient Descent

Backpropagation computes gradients of the loss function with respect to network parameters using the chain rule. These gradients are then used by Gradient Descent to update weights and biases, allowing the network to learn from training data and reduce prediction error.

7. Training Procedure

The neural network was trained using the Iris dataset after preprocessing and standardization. Training was performed for 1000 epochs using Gradient Descent optimization with a learning rate of 0.1.

During each epoch, the following steps were executed:

1. Forward Propagation was performed to generate predictions.
2. Cross-Entropy Loss was calculated to measure prediction error.
3. Backpropagation was used to compute gradients for all network parameters.
4. Weights and biases were updated using Gradient Descent.
5. Validation accuracy was monitored periodically to evaluate model performance.

The training process continued until the specified number of epochs was completed. The model successfully learned the underlying patterns in the dataset and achieved high classification accuracy on unseen test data.

8. Results and Discussion

The Artificial Neural Network implemented from scratch was successfully trained and evaluated on the Iris dataset. The model achieved excellent classification performance and demonstrated the effectiveness of neural networks for multiclass classification tasks.

Model Performance

Metric	Value
Test Accuracy	100%
Epochs	1000
Learning Rate	0.1
Dataset	Iris Dataset

Activation Function Comparison

Activation Function Test Accuracy

Sigmoid	100.00%
Tanh	95.65%

The Sigmoid activation function achieved the highest accuracy for the selected architecture. Although both activation functions performed well, Sigmoid provided slightly better classification results on the Iris dataset.

Visual Analysis

The training loss curve showed a consistent decrease in loss throughout the training process, indicating successful learning by the neural network. The training and validation accuracy curves demonstrated stable model performance without significant overfitting.

The confusion matrix showed that most test samples were correctly classified into their respective classes. Additionally, PCA visualization revealed clear separation among the three Iris flower species, which contributed to the high classification accuracy achieved by the model.

Discussion

The results indicate that a neural network implemented from scratch can achieve excellent performance when appropriate preprocessing, parameter initialization, and optimization techniques are applied.

9. Conclusion

In this project, an Artificial Neural Network (ANN) was successfully implemented from scratch using NumPy for the classification of Iris flower species. The implementation included data preprocessing, forward propagation, backpropagation, cross-entropy loss computation, and gradient descent optimization. The model was trained and evaluated on the Iris dataset and achieved a test accuracy of 100%. The project provided practical experience in understanding the internal working of neural

networks rather than relying on high-level deep learning frameworks. Through this implementation, concepts such as activation functions, weight updates, gradient computation, and model evaluation were explored in detail. Overall, the assignment helped build a strong foundation in neural network fundamentals and deep learning concepts.

